PYTHON PROGRAMS

Algorithms • Manual Flowcharts • Code • Output

A student practical record containing 10 Python programs. Each program includes an algorithm, a flowchart made with standard shapes, Python code, and sample output.

Flowchart symbols: Oval = Start/Stop • Parallelogram = Input/Output • Rectangle = Process • Diamond = Decision

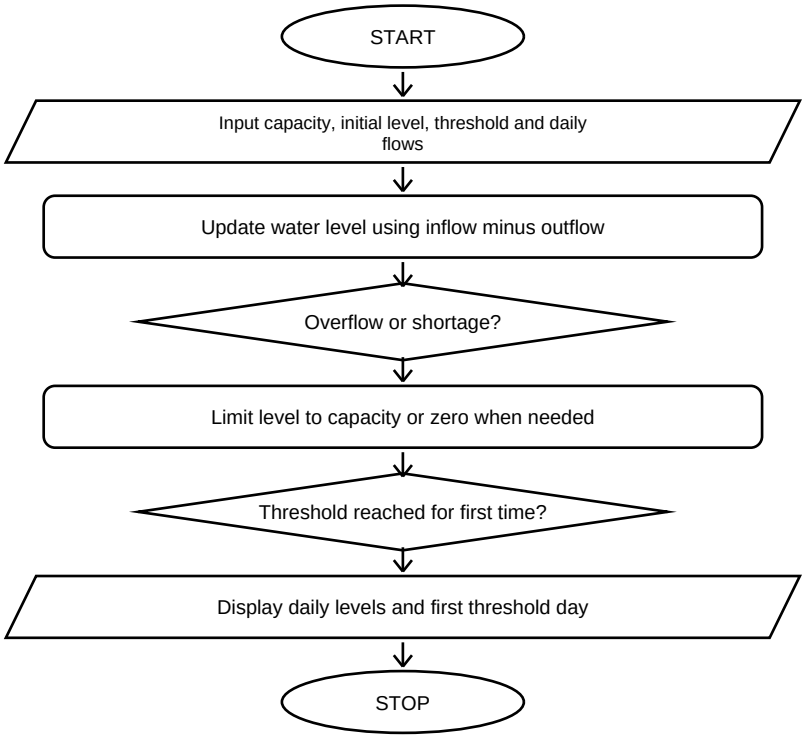
1. Multi-Day Water Tank Simulation

Algorithm

1. Start with the initial water level, tank capacity, and threshold.
2. Read the daily inflow and outflow quantities.
3. Update water level = previous level + inflow – outflow.
4. If level exceeds capacity, report overflow and set level to capacity.
5. If level is below zero, report shortage and set level to zero.
6. Record the first day the threshold is reached.
7. Repeat for all days, display results, and stop.

Flowchart

Standard flowchart symbols are drawn below.



Python Code

```
capacity = 100
level = 60
threshold = 80

inflow = [30, 10, 5, 20, 15]
outflow = [10, 20, 15, 10, 25]
first_day = None

for day in range(len(inflow)):
    level += inflow[day] - outflow[day]

    if level > capacity:
        print(f"Day {day + 1}: Overflow")
        level = capacity
    elif level < 0:
        print(f"Day {day + 1}: Shortage")
        level = 0

    print(f"Day {day + 1}: Water level = {level}")

    if first_day is None and level >= threshold:
        first_day = day + 1
```

```
if first_day is not None:
    print("Threshold first reached on day:", first_day)
else:
    print("Threshold not reached")
```

Sample Output

```
Day 1: Water level = 80
Day 2: Water level = 70
Day 3: Water level = 60
Day 4: Water level = 70
Day 5: Water level = 60
Threshold first reached on day: 1
```

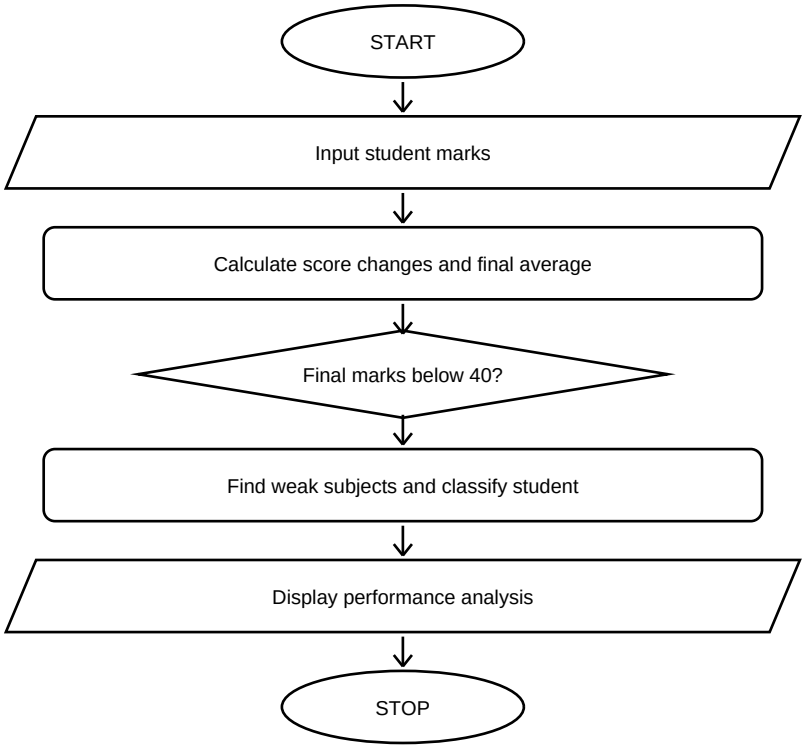
2. Student Performance Trend Analysis

Algorithm

1. Store each student's marks for three examinations.
2. Calculate the change between the first and last examination.
3. Identify subjects whose final marks are below 40.
4. Calculate the final average and classify the student.
5. Display changes, weak subjects, average, and classification.

Flowchart

Standard flowchart symbols are drawn below.



Python Code

```
students = {
    "Anu": {
        "Maths": [60, 70, 85],
        "Science": [55, 65, 75],
        "English": [35, 38, 39]
    },
    "Ravi": {
        "Maths": [80, 75, 70],
        "Science": [70, 65, 60],
        "English": [75, 70, 65]
    }
}

for name, subjects in students.items():
    marks = []
    weak = []
    print("\\nStudent:", name)

    for subject, scores in subjects.items():
        change = scores[-1] - scores[0]
        marks.append(scores[-1])
        print(subject, "change:", change)

        if scores[-1] < 40:
            weak.append(subject)
    average = sum(marks) / len(marks)
```

```
    if average >= 75:
        category = "Excellent"
    elif average >= 60:
        category = "Good"
    elif average >= 40:
        category = "Average"
    else:
        category = "Needs Improvement"

    print("Final average:", round(average, 2))
    print("Weak subjects:", weak if weak else "None")
    print("Classification:", category)
```

Sample Output

```
Student: Anu
Maths change: 25
Science change: 20
English change: 4
Final average: 66.33
Weak subjects: ['English']
Classification: Good

Student: Ravi
Maths change: -10
Science change: -10
English change: -10
Final average: 65.0
Weak subjects: None
Classification: Good
```

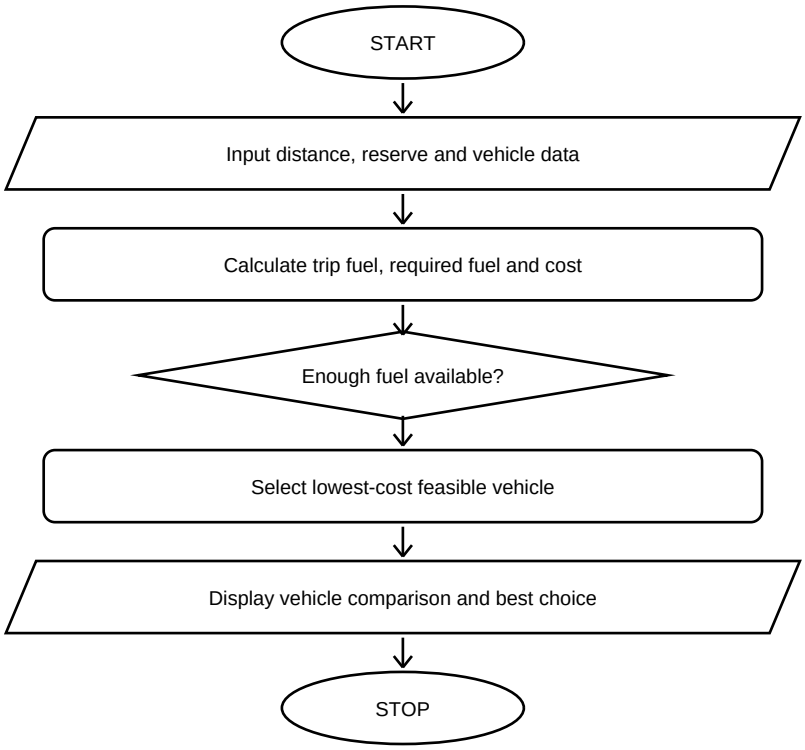
3. Fuel Consumption and Route Comparison

Algorithm

1. Input journey distance and minimum reserve fuel.
2. Store each vehicle's mileage, fuel price, and available fuel.
3. Calculate trip fuel, required fuel including reserve, and cost.
4. Check whether each vehicle has enough fuel.
5. Choose the feasible vehicle with the lowest journey cost.
6. Display the results.

Flowchart

Standard flowchart symbols are drawn below.



Python Code

```
distance = 300  # km
reserve = 5     # litres

vehicles = {
    "Car A": {"mileage": 15, "price": 105,
              "available_fuel": 30},
    "Car B": {"mileage": 20, "price": 105,
              "available_fuel": 25},
    "Car C": {"mileage": 12, "price": 100,
              "available_fuel": 40}
}

best_vehicle = None
lowest_cost = float("inf")

for name, data in vehicles.items():
    trip_fuel = distance / data["mileage"]
    required = trip_fuel + reserve
    cost = trip_fuel * data["price"]
    feasible = data["available_fuel"] >= required

    print(f"\\n{name}")
    print("Trip fuel:", round(trip_fuel, 2), "litres")
    print("Required with reserve:", round(required, 2))
```

```
    print("Journey cost: Rs.", round(cost, 2))
    print("Feasible:", feasible)

    if feasible and cost < lowest_cost:
        lowest_cost = cost
        best_vehicle = name

if best_vehicle:
    print("\\nLowest-cost feasible vehicle:", best_vehicle)
    print("Minimum journey cost: Rs.", round(lowest_cost, 2))
else:
    print("\\nNo feasible vehicle")
```

Sample Output

```
Car A
Trip fuel: 20.0 litres
Required with reserve: 25.0
Journey cost: Rs. 2100.0
Feasible: True

Car B
Trip fuel: 15.0 litres
Required with reserve: 20.0
Journey cost: Rs. 1575.0
Feasible: True

Car C
Trip fuel: 25.0 litres
Required with reserve: 30.0
Journey cost: Rs. 2500.0
Feasible: True

Lowest-cost feasible vehicle: Car B
Minimum journey cost: Rs. 1575.0
```

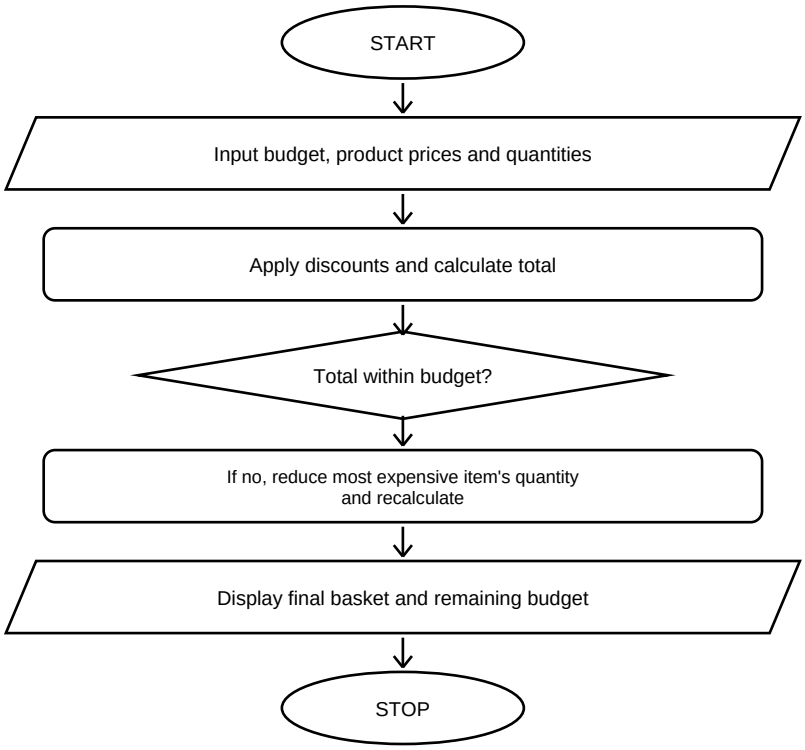
4. Multi-Product Shopping Budget Constraint

Algorithm

1. Set the available budget and store product prices and quantities.
2. Calculate each product subtotal and apply eligible discounts.
3. Calculate the total basket cost.
4. If the total exceeds budget, reduce the quantity of the most expensive available item and recalculate.
5. Repeat until the total is within budget.
6. Display the final basket and remaining budget.

Flowchart

Standard flowchart symbols are drawn below.



Python Code

```
budget = 1000

products = {
    "Rice": {"price": 300, "qty": 2},
    "Oil": {"price": 180, "qty": 2},
    "Soap": {"price": 50, "qty": 4}
}

def calculate_total():
    total = 0
    for name, item in products.items():
        subtotal = item["price"] * item["qty"]
        if name == "Rice" and item["qty"] >= 2:
            subtotal *= 0.90
        total += subtotal
    return total

print("Initial total:", calculate_total())

while calculate_total() > budget:
    candidates = [
        (item["price"], name)
        for name, item in products.items()
        if item["qty"] > 0
```

```
    ]
    if not candidates:
        break
    _, name = max(candidates)
    products[name]["qty"] -= 1
    print("Reduced:", name)

print("\\nFinal basket:")
for name, item in products.items():
    print(name, "quantity:", item["qty"])

print("Final total:", round(calculate_total(), 2))
print("Remaining budget:", round(budget - calculate_total(), 2))
```

Sample Output

```
Initial total: 1000.0

Final basket:
Rice quantity: 2
Oil quantity: 2
Soap quantity: 4
Final total: 1000.0
Remaining budget: 0.0
```

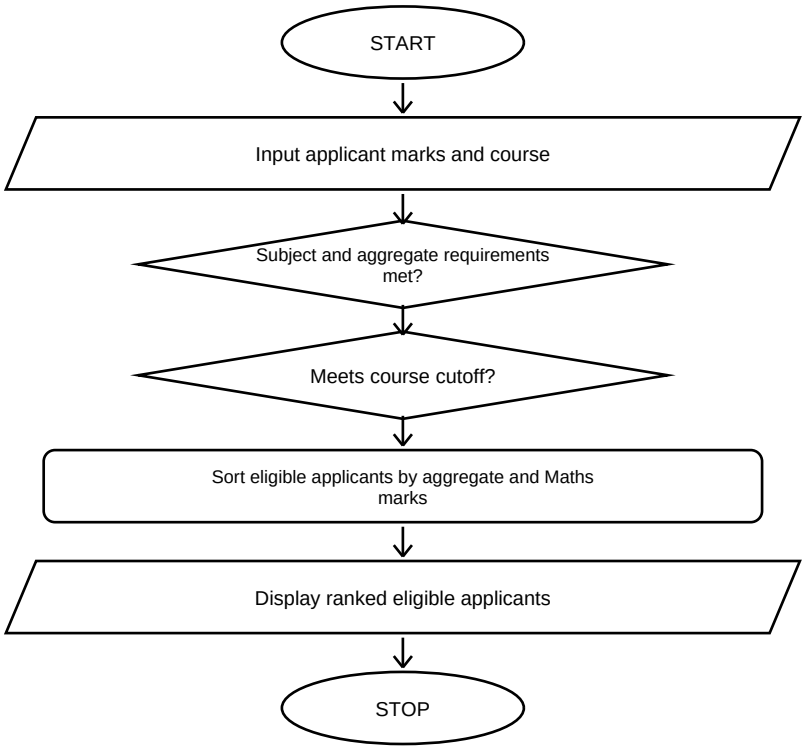
5. University Admission Decision Engine

Algorithm

1. Read applicants' subject marks and aggregate marks.
2. Check minimum marks in each required subject.
3. Check minimum aggregate and course cutoff.
4. Keep eligible applicants and report rejected applicants.
5. Sort eligible applicants by aggregate, then mathematics marks, in descending order.
6. Display the ranked list.

Flowchart

Standard flowchart symbols are drawn below.



Python Code

```
applicants = [
    {"name": "Anu", "maths": 90, "physics": 85,
     "chemistry": 80, "aggregate": 85},
    {"name": "Bala", "maths": 95, "physics": 80,
     "chemistry": 75, "aggregate": 85},
    {"name": "Chitra", "maths": 70, "physics": 65,
     "chemistry": 70, "aggregate": 68},
    {"name": "Deepak", "maths": 88, "physics": 90,
     "chemistry": 86, "aggregate": 88}
]

minimum_subject = 50
minimum_aggregate = 60
course_cutoff = 70
eligible = []

for student in applicants:
    subject_ok = (
        student["maths"] >= minimum_subject
        and student["physics"] >= minimum_subject
        and student["chemistry"] >= minimum_subject
    )
    aggregate_ok = student["aggregate"] >= minimum_aggregate
    cutoff_ok = student["aggregate"] >= course_cutoff
```

```
    if subject_ok and aggregate_ok and cutoff_ok:
        eligible.append(student)
    else:
        print("Not eligible:", student["name"])

eligible.sort(
    key=lambda s: (s["aggregate"], s["maths"]),
    reverse=True
)

print("\\nRanked eligible applicants:")
for rank, student in enumerate(eligible, start=1):
    print(rank, student["name"], student["aggregate"])
```

Sample Output

```
Not eligible: Chitra

Ranked eligible applicants:
1 Deepak 88
2 Bala 85
3 Anu 85
```

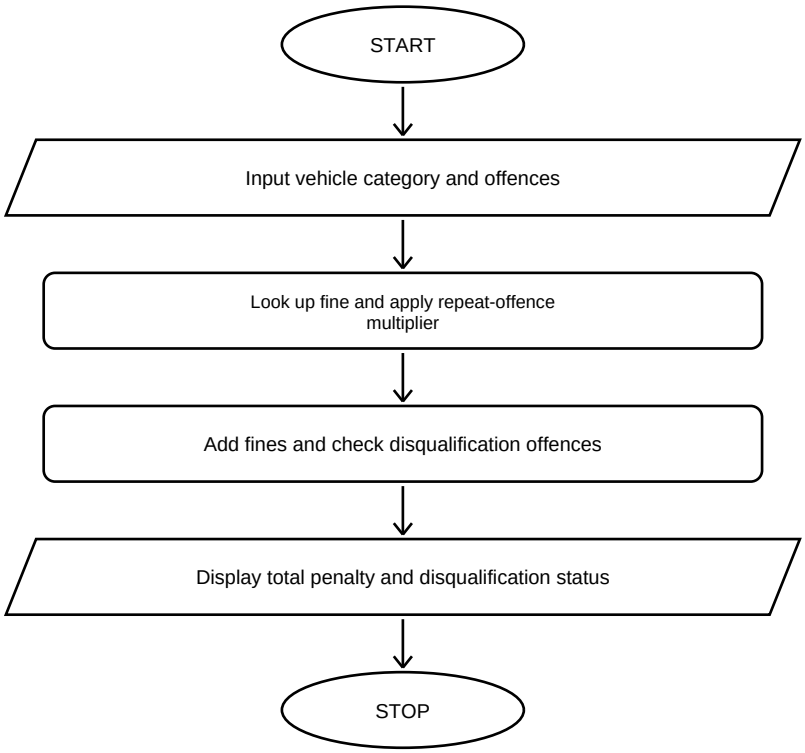
6. Traffic Violation Penalty Calculator

Algorithm

1. Store fine amounts for each offence and vehicle category.
2. Input the vehicle type and offences.
3. Look up the fine for each offence.
4. Double the fine for a repeat offence and add it to the total.
5. Check whether any offence requires disqualification.
6. Display each fine, total penalty, and disqualification status.

Flowchart

Standard flowchart symbols are drawn below.



Python Code

```
fine_rules = {
    "car": {
        "speeding": 1000,
        "no_helmet": 0,
        "signal_jump": 1500,
        "drunk_driving": 10000
    },
    "motorcycle": {
        "speeding": 500,
        "no_helmet": 1000,
        "signal_jump": 1000,
        "drunk_driving": 10000
    }
}

disqualifying_offences = {"drunk_driving"}
vehicle = "car"
violations = [
    ("speeding", False),
    ("signal_jump", True),
    ("drunk_driving", False)
]

total = 0
```

```
disqualified = False

for offence, repeat in violations:
    fine = fine_rules[vehicle][offence]
    if repeat:
        fine *= 2
    total += fine
    print(offence, "- Fine: Rs.", fine)

    if offence in disqualifying_offences:
        disqualified = True

print("\\nTotal penalty: Rs.", total)
print("Disqualification:", "Yes" if disqualified else "No")
```

Sample Output

```
speeding - Fine: Rs. 1000
signal_jump - Fine: Rs. 3000
drunk_driving - Fine: Rs. 10000

Total penalty: Rs. 14000
Disqualification: Yes
```

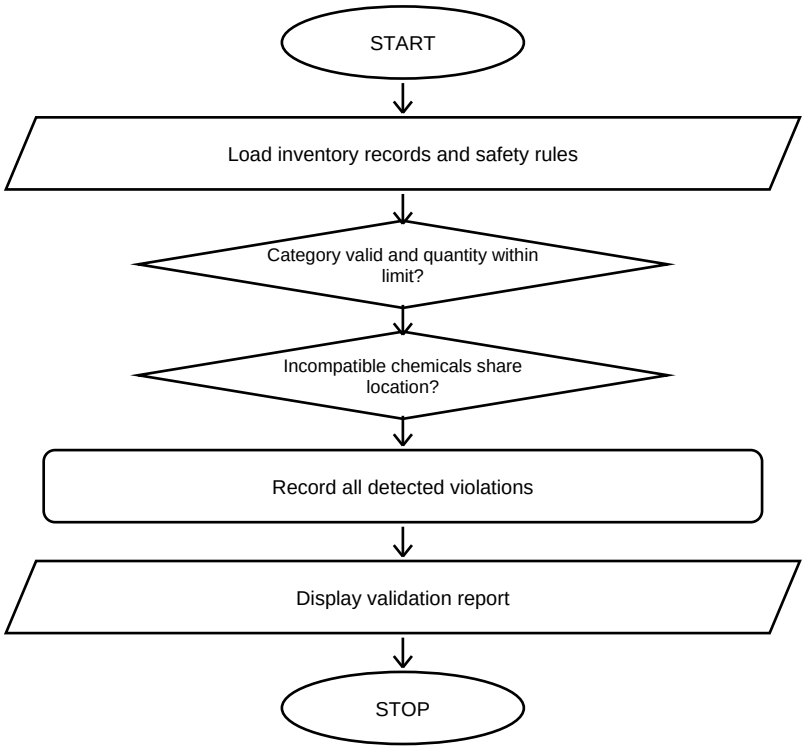
7. Laboratory Chemical Inventory Validation

Algorithm

1. Define maximum quantities for storage categories.
2. Define pairs of incompatible chemicals.
3. Read inventory records.
4. Check each chemical's storage category and quantity limit.
5. Compare chemicals in the same location against incompatibility rules.
6. Report all detected violations or confirm that the inventory is valid.

Flowchart

Standard flowchart symbols are drawn below.



Python Code

```
max_quantity = {
    "flammable": 50,
    "corrosive": 30,
    "oxidizer": 20
}

inventory = [
    {"name": "Ethanol", "category": "flammable",
     "quantity": 40, "location": "A"},
    {"name": "Acid", "category": "corrosive",
     "quantity": 35, "location": "B"},
    {"name": "Oxidizer", "category": "oxidizer",
     "quantity": 15, "location": "A"}
]

incompatible = {
    frozenset(["Ethanol", "Oxidizer"])
}
issues = []

for item in inventory:
    category = item["category"]
    if category not in max_quantity:
        issues.append(f"{item['name']}: Invalid storage category")
```

```
    elif item["quantity"] > max_quantity[category]:
        issues.append(f"{item['name']}: Quantity exceeds limit")

for i in range(len(inventory)):
    for j in range(i + 1, len(inventory)):
        a, b = inventory[i], inventory[j]
        pair = frozenset([a["name"], b["name"]])
        if a["location"] == b["location"] and pair in incompatible:
            issues.append(
                f"{a['name']} and {b['name']}: Incompatible storage"
            )

if issues:
    print("Inventory validation failed:")
    for issue in issues:
        print("-", issue)
else:
    print("Inventory is valid")
```

Sample Output

```
Inventory validation failed:
- Acid: Quantity exceeds limit
- Ethanol and Oxidizer: Incompatible storage
```

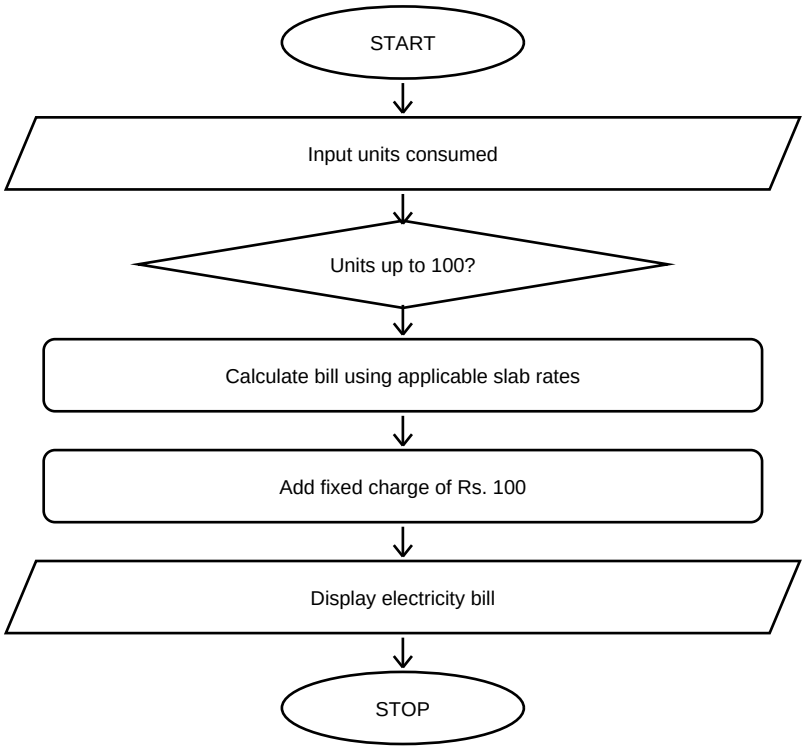
8. Progressive Electricity Billing

Algorithm

1. Start and read units consumed.
2. If units are up to 100, charge ■1.50 per unit.
3. If units are 101–200, charge ■1.50 for the first 100 units and ■2.50 for remaining units.
4. If units exceed 200, add ■4 per unit above 200 to the first two slab charges.
5. Add the fixed charge of ■100.
6. Display the bill and stop.

Flowchart

Standard flowchart symbols are drawn below.



Python Code

```
units = int(input("Enter units consumed: "))

if units <= 100:
    bill = units * 1.5
elif units <= 200:
    bill = (100 * 1.5) + (units - 100) * 2.5
else:
    bill = (100 * 1.5) + (100 * 2.5) + (units - 200) * 4

fixed_charge = 100
bill = bill + fixed_charge
print("Electricity Bill = Rs.", bill)
```

Sample Output

```
Enter units consumed: 250
Electricity Bill = Rs. 700.0
```

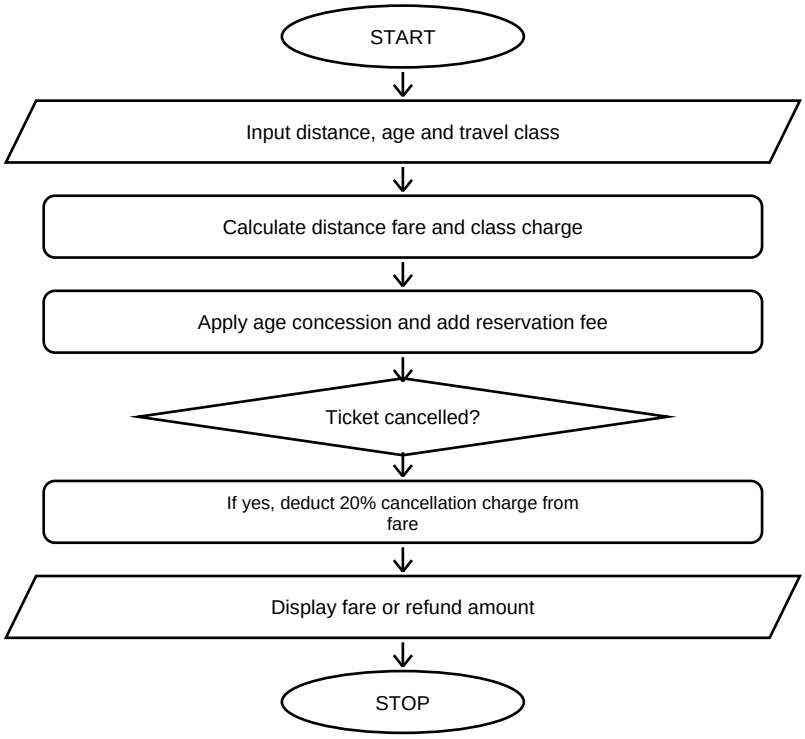
9. Railway Fare and Reservation Calculator

Algorithm

1. Input distance, age, and travel class.
2. Calculate the distance-based fare using the given rate rules.
3. Add the Sleeper or AC class charge.
4. Apply the age concession: 50% below age 12 or 30% above age 60.
5. Add the reservation fee of ■50.
6. Ask whether the ticket is cancelled; if yes, calculate the refund after a 20% cancellation charge.
7. Display fare details and stop.

Flowchart

Standard flowchart symbols are drawn below.



Python Code

```
distance = int(input("Enter distance (km): "))
age = int(input("Enter age: "))
travel_class = input("Enter class (Sleeper/AC): ")

if distance <= 100:
    fare = distance * 1
elif distance <= 300:
    fare = distance * 1.5
else:
    fare = distance * 2

if travel_class.lower() == "ac":
    fare += 300
else:
    fare += 100

if age < 12:
    fare = fare * 0.5
elif age > 60:
    fare = fare * 0.7

fare += 50
```

```
print("Total Fare = Rs.", fare)

cancel = input("Cancel ticket? (yes/no): ")
if cancel.lower() == "yes":
    refund = fare * 0.8
    print("Refund Amount = Rs.", refund)
```

Sample Output

```
Example 1:
Enter distance (km): 135
Enter age: 18
Enter class (Sleeper/AC): sleeper
Total Fare = Rs. 352.5
Cancel ticket? (yes/no): no

Example 2:
Enter distance (km): 350
Enter age: 66
Enter class (Sleeper/AC): AC
Total Fare = Rs. 750.0
Cancel ticket? (yes/no): no
```

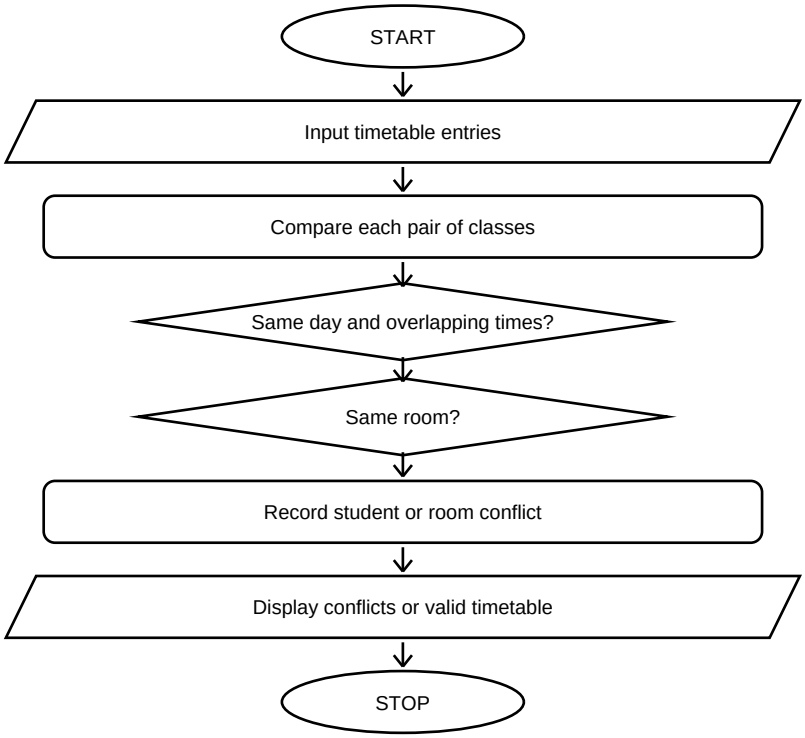
10. Student Timetable Conflict Detection

Algorithm

1. Store each class as course, day, start time, end time, and room.
2. Compare each pair of classes.
3. Check whether the classes are on the same day and their time intervals overlap.
4. If overlapping classes use different rooms, report a student conflict.
5. If overlapping classes use the same room, report a room conflict.
6. Display all conflicts or confirm that the timetable is valid.

Flowchart

Standard flowchart symbols are drawn below.



Python Code

```
# (course, day, start, end, room)
timetable = [
    ("Maths", "Monday", 9, 10, "R1"),
    ("Physics", "Monday", 9, 11, "R2"),
    ("English", "Monday", 11, 12, "R1"),
    ("Chemistry", "Tuesday", 10, 12, "R1")
]

conflicts = []

for i in range(len(timetable)):
    for j in range(i + 1, len(timetable)):
        a = timetable[i]
        b = timetable[j]
        same_day = a[1] == b[1]
        overlap = a[2] < b[3] and b[2] < a[3]

        if same_day and overlap:
            if a[4] == b[4]:
                conflicts.append(
                    f"Room conflict: {a[0]} and {b[0]}"
                )
            else:
                conflicts.append(
```

```
                    f"Student conflict: {a[0]} and {b[0]}"
                )

if conflicts:
    print("Timetable is invalid:")
    for conflict in conflicts:
        print("-", conflict)
else:
    print("Timetable is valid")
```

Sample Output

```
Timetable is invalid:
- Student conflict: Maths and Physics
```